Programación Multimedia y Dispositivos Móviles

2º DAM — Unidad 1: Tecnologías para aplicaciones en dispositivos móviles

1. Introducción

En este módulo vamos a ver cómo se hacen aplicaciones para el móvil, tanto "normales" como con contenido multimedia (audio, vídeo, imágenes...). El curso lo he organizado en cuatro bloques: hoy empezamos con las tecnologías que existen para hacer apps móviles, luego pasaremos a programar en Android con Kotlin, más adelante veremos Flutter (que sirve para Android e iOS con el mismo código), y acabaremos el curso metiéndonos con videojuegos usando Unity.

Para hoy el objetivo es sencillo: que os hagáis una idea de qué tipos de apps existen, por qué programar para un móvil no es lo mismo que para un ordenador, y que dejemos instalado Android Studio, que va a ser nuestra herramienta de trabajo durante buena parte del curso.

2. Tipos de tecnologías para aplicaciones móviles

A la hora de hacer una app hay que decidir con qué enfoque se va a construir, porque no todos valen para lo mismo. Básicamente hay tres caminos, y cada uno tiene sus ventajas y sus problemas.

2.1. Aplicaciones nativas

Es programar la app directamente para un sistema operativo concreto, usando sus herramientas oficiales: Kotlin o Java si es para Android, Swift si es para iPhone. Es la opción que más rendimiento da y con la que se puede acceder a todo el hardware del móvil (cámara, GPS, sensores...), pero tiene una pega gorda: si quieres la app tanto en Android como en iPhone, la tienes que programar dos veces, prácticamente desde cero cada una.

2.2. Aplicaciones híbridas o multiplataforma

Aquí el código se escribe una sola vez y hay un framework que se encarga de que funcione tanto en Android como en iOS. Los más usados ahora mismo son Flutter, de Google (con el lenguaje Dart), y React Native, de Meta (con JavaScript). Van muy bien de rendimiento, casi como una nativa, y te ahorras tener que programar la app dos veces. De hecho Flutter es uno de los bloques que veremos más adelante en este mismo módulo.

2.3. Aplicaciones web o PWA

Estas ni se instalan: se abren dentro del navegador del móvil, igual que cualquier página web, hechas con HTML, CSS y JavaScript. Una PWA (Progressive Web App) le añade algún truco para que se comporte un poco más como una app de verdad, como poner un icono en el escritorio o funcionar algo sin conexión, pero el acceso al hardware del móvil sigue siendo muy limitado.

2.4. Comparativa

	Apps nativas	Apps híbridas	Apps web / PWA
Lenguaje / tecnología	Kotlin, Java (Android); Swift (iOS)	Dart+Flutter, JavaScript+React Native	HTML, CSS, JavaScript
Rendimiento	Máximo (acceso directo al SO)	Muy bueno, algo menor que nativo	Limitado, depende del navegador
Acceso al hardware	Total (cámara, GPS, sensores...)	Amplio, vía plugins	Muy limitado
Un código para varias plataformas	No (hay que programar cada una)	Sí, un único proyecto	Sí, siempre
Distribución	Google Play / App Store	Google Play / App Store	Solo con navegador o URL
Ejemplo real	Spotify, WhatsApp	Google Ads, Alibaba	Twitter Lite

3. Por qué programar para móvil es distinto

Un ordenador de sobremesa y un móvil no se parecen tanto como pensáis a la hora de programar. Hay una serie de cosas que siempre hay que tener en la cabeza cuando se hace una app:

- La batería: cualquier cosa que hagáis (usar el GPS, tener la pantalla encendida, conectarse a internet) gasta batería, y una app mal hecha puede dejar el móvil sin batería en un rato.
- Poca memoria y poco espacio: un móvil tiene mucha menos RAM y almacenamiento que un ordenador, y si una app consume demasiado, el propio sistema la cierra sin avisar.
- Pantallas de todos los tamaños: no todos los móviles tienen la misma pantalla, así que la interfaz tiene que adaptarse a tamaños, resoluciones y a que gires el móvil.
- Te puedes quedar sin conexión en cualquier momento: en un túnel, en un ascensor, con el modo avión puesto... la app tiene que saber qué hacer en ese caso.
- No hay teclado ni ratón: se maneja todo con los dedos, con gestos, y además el móvil tiene sensores que un ordenador no tiene (GPS, acelerómetro...).
- Los permisos y la seguridad: si la app quiere tocar algo sensible (contactos, ubicación, cámara), tiene que pedírselo al usuario primero.

4. Con qué vamos a trabajar: el IDE

Un IDE es, para que os hagáis una idea, el "programa madre" donde vais a escribir el código, probarlo y ver los errores, todo en un mismo sitio. Trae el editor, un simulador de móvil para probar la app sin tener uno a mano, y todo lo necesario para que funcione.

Android Studio: es el que vamos a usar nosotros, es el oficial de Google para hacer apps de Android y con el que vamos a trabajar la mayor parte del curso. Más adelante veremos también Flutter y, para la parte de videojuegos, Unity, pero hoy toca instalar y familiarizarnos con Android Studio.

5. Instalamos Android Studio

Vamos a hacerlo ahora mismo, todos a la vez, paso a paso. La descarga ocupa varios GB, así que en cuanto le deis a descargar podemos seguir hablando de teoría mientras baja.

Requisitos mínimos

- Windows 10/11 (64 bits), macOS 12+ o Linux — 8 GB de RAM recomendados (4 GB mínimo)
- Al menos 8 GB de espacio libre en disco
- Conexión a internet estable para la descarga

Pasos a seguir

1. Abrid el navegador y entrad en developer.android.com/studio
2. Pulsad el botón de descarga ("Download Android Studio") y aceptad los términos.
3. Una vez descargado, ejecutad el instalador y aceptad las opciones por defecto (instalación "Standard").
4. El asistente descargará automáticamente el SDK de Android y creará un dispositivo virtual (emulador). Esto puede tardar varios minutos: es normal.
5. Cuando termine, se abrirá la pantalla de bienvenida de Android Studio ("Welcome to Android Studio").
6. Pulsad en "New Project", elegid la plantilla "Empty Activity" y ponedle de nombre MiPrimeraApp.
7. Dejad el lenguaje en Kotlin y pulsad "Finish". Android Studio abrirá el proyecto (puede tardar la primera vez, está indexando).

8. Para comprobar que todo funciona, pulsad el botón verde de "Run" (▶) y elegid el emulador que se creó antes. Si aparece la pantalla con el texto "Hello World!", la instalación ha ido bien.

6. Lo que quiero que os llevéis de hoy

Esto lo hacemos todo aquí, en clase, no hace falta tocar nada en casa. Al acabar la sesión de hoy quiero que cada uno tenga, en su propio ordenador:

- Android Studio instalado y funcionando.
- Un proyecto creado, llamado MiPrimeraApp.
- La app ejecutada al menos una vez en el emulador, viendo en pantalla el típico "Hello World!".

Si a alguien le sobra tiempo, que pruebe a cambiar el texto "Hello World!" por su nombre en el archivo activity_main.xml y a ejecutar otra vez la app para ver cómo cambia. No pasa nada si no sale a la primera, es solo para perder el miedo a tocar el código.